Build a complete pgTAP test suite covering all 10 OWASP ASVS controls for Supabase RLS policies — with copy-paste assertions and a ready-to-run GitHub Actions CI workflow

Computer
AppSec debugging runbook: 10 student-app vulnerabilities mapped to Supabase RLS bypasses and pgTAP failure assertions
Computer
Show me the rest of the 9 OWASP ASVS pgTAP assertions
Show me how to configure the GitHub Actions workflow for Supabase CI
Show me the Jest JWT validation tests code

Created the complete Supabase-focused AppSec package: one pgTAP suite covering all 10 authorization/RLS control families, JWT Jest tests, a ready-to-adapt GitHub Actions workflow, and a vulnerability-to-failure-assertion runbook. The suite is intentionally a **schema-adaptation template**: replace the illustrative tables, UUIDs, helpers, expected policy names, and function signatures before enabling it in CI.

Files created

File	Recommended location	Purpose
`20261006_asvs_rls_suite.test.sql`	`supabase/tests/`	34 pgTAP assertions across the 10 Supabase RLS/ASVS control families
`authTokens.test.ts`	`app/src/security/`	Jest/Vitest-compatible JWT validation tests
`supabase-security-ci.yml`	`.github/workflows/`	Local Supabase startup, migration reset, pgTAP, Jest, SAST, dependency audit, secrets, and workflow-pin checks
`ASVS-RLS-10-VULNERABILITY-FAILURE-MAP.md`	`docs/assurance/`	Ten student-app vulnerabilities mapped to RLS failures, pgTAP assertions, and remediation controls
`semester-complete-pgtap-asvs-suite.zip`	—	Bundle of all four files

The nine additional pgTAP control families

AC-03 was your core cross-tenant test. The new complete suite adds these other nine controls:

Control	ASVS area	pgTAP assertion behavior
C02	V8 Authorization	A student cannot insert a record with another tenant's `tenant_id`
C03	V8 Authorization	A student cannot self-insert role grants or take over `owner_id`
C04	V8 Authorization	A direct privileged RPC cannot return a foreign resource
C05	V8 Authorization	Views, derived data, and exports do not leak a foreign tenant's record
C06	V8 Authorization	A student cannot enumerate private storage metadata for a foreign tenant
C07	V8 Authorization	Suspended or revoked users cannot access protected records
C08	V8 Authorization	Expired staff grants do not authorize access
C09	V8 Authorization	Privileged/RPC functions have deliberately restricted execution roles
C10	V8 + V13	RLS is enabled and anonymous callers cannot read protected records

Each has an expected **failure assertion**. For example, a spoofed cross-tenant insertion should fail:

```
select throws_ok(
  $$ insert into app.student_records (id, tenant_id, owner_id, display_name)
     values (
       '66666666-6666-6666-6666-666666666666',
       'tenant-t2-id',
       'student-a-id',
       'spoofed T2 row'
     ) $$,
  'new row violates row-level security policy for table "student_records"',
  'student A cannot insert a record into T2'
);
```

A blocked update commonly produces zero affected rows rather than an exception:

```
select results_eq(
  $$ with x as (
       update app.student_records
       set display_name = 'blocked-T2-write'
       where id = 'record-owned-by-T2'
       returning id
     )
     select count(*) from x $$,
  $ values (0::bigint) $,
  'student A cannot update T2 record'
);
```

The suite also includes a **positive control**—Student A can access an allowed T1 record—and verifies forbidden changes did not occur after switching back to the fixture-verifier role. This

prevents false passes caused by missing fixtures or a policy that denies everyone.

pgTAP supports assertions including `results_eq`, `throws_ok`, `lives_ok`, policy checks, privilege checks, and schema checks. Supabase's CLI executes pgTAP files from `supabase/tests`, and a local stack must be running before `supabase test db --local`.[1] [2]

Configure Supabase CI

Place the generated workflow here:

```
.github/workflows/supabase-security-ci.yml
```

Then replace all placeholders before enabling it:

```
REPLACE_WITH_REVIEWED_COMMIT_SHA
REPLACE_WITH_REVIEWED_CLI_VERSION
REPLACE_WITH_REVIEWED_IMAGE_DIGEST
```

The workflow sequence is:

```
- uses: supabase/setup-cli@REVIEWED_SHA
  with:
    version: PINNED_CLI_VERSION

- run: supabase start
- run: supabase db reset --local
- run: supabase test db --local
```

`supabase db reset --local` recreates the local database and reapplies committed migrations; it can also load `supabase/seed.sql` if you use one. The test command executes the pgTAP files in `supabase/tests`.[3] [1]

The generated workflow has four jobs:

1. `pgtap-rls` — starts an isolated local stack and runs the RLS suite.
2. `application-security` — runs type checking, Jest/Vitest, SAST, and dependency auditing.
3. `secrets-and-workflow` — scans Git history and rejects mutable GitHub Action tags.
4. `merge-security-gate` — succeeds only when all prior security jobs succeed.

Add these as required status checks in GitHub branch protection:

```
Migrations and pgTAP RLS suite
Jest and static security tests
Secret scan and workflow policy
Required security gate
```

The workflow intentionally does not need production Supabase credentials for the local pgTAP suite. Keep staging and production migration deployment workflows separate from test

workflows.

Jest JWT validation code

The generated test file uses `jose` and asserts:

- Correct signature.
- Allowed `HS256` algorithm in the unit-test fixture.
- Expected issuer.
- Expected audience.
- Non-expired token.
- Maximum token age.
- Required `sub` claim.
- Required application role.
- Rejection of another signing key.

Core verification code:

```
const { payload } = await jwtVerify(token, testKey, {
  algorithms: ["HS256"],
  issuer,
  audience,
  currentDate: now,
  maxTokenAge: "60m",
  clockTolerance: 0,
});

if (!payload.sub) throw new Error("missing subject");
if (payload.role !== "authenticated") {
  throw new Error("invalid role");
}
```

The test key is deliberately ephemeral and only for local unit tests. Replace the production implementation with your trusted issuer/JWKS or Supabase verification configuration—never embed a production signing secret in frontend, test code, or GitHub Actions. JWT verification should validate the signature and claims such as issuer, audience, subject, and expiration. [4] [5]

Important adaptation checklist

Before committing the suite:

- Replace `app.student_records`, `app.role_grants`, view names, function names, storage bucket names, policy names, and UUIDs.
- Implement a deterministic synthetic fixture procedure; do not run with production data.
- Confirm the exact expected RLS error strings for your installed PostgreSQL/Supabase version.

- Add individualized direct-call tests for every `SECURITY DEFINER` function.
- Add API integration tests for all database tests, since direct SQL does not exercise your server/API authorization layer.
- Add signed-URL, realtime, Edge Function, worker, cache, and export-path tests where those features exist.
- Replace all placeholder action SHA pins and container digests.
- Enable required branch protection checks; a workflow alone cannot block merging.

⁂

1. https://supabase.com/docs/reference/cli/introduction
2. https://pigsty.io/ext/e/pgtap/
3. https://supabase.com/docs/guides/deployment/ci/testing
4. https://github.com/panva/jose/blob/main/docs/jwt/verify/functions/jwtVerify.md
5. https://github.com/panva/jose/blob/main/docs/jwt/verify/interfaces/JWTVerifyOptions.md
6. https://github.com/orgs/supabase/discussions/8950
7. https://github.com/orgs/supabase/discussions/35771
8. https://stackoverflow.com/questions/78188818/cant-get-variables-with-supabase-status-on-github-actions
9. https://hexdocs.pm/gose/2.0.0/gose/jose/jwt.html
10. https://connect2id.com/products/nimbus-jose-jwt/examples/validating-jwt-access-tokens
11. https://stackoverflow.com/questions/66823618/is-options-ignoreexpiration-still-valid-in-node-jose
12. https://supabase.com/docs/guides/deployment/managing-environments
13. https://www.reddit.com/r/Supabase/comments/1r7h9wc/local_database_for_development_versus_automated/
14. https://github.com/supabase/cli/issues/45
15. https://github.com/orgs/supabase/discussions/16415